Q. What key shift in developer behaviour drove this change?

A. The biggest shift has been the rise of open source, which encouraged developers to collaborate, contribute, and build across software, documentation, education, and AI projects. India now ranks second globally in open source AI contributions. At the same time, AI and agentic AI have made software creation accessible beyond traditional coders, enabling people from non-software backgrounds to solve real-world problems using tools like GitHub Copilot. This has democratised software development, accelerated collaboration, and significantly increased innovation and problem-solving through technology.

Q. How has AI changed the understanding of developer productivity?

A. Agentic AI is shifting developers from being coders to problem solvers and orchestrators of intelligent agents. Instead of manually writing every layer of software, developers now guide specialised AI agents across frontend, backend, databases, and testing while focusing more on creativity, judgment, accessibility, and real world impact. This has significantly improved productivity by enabling faster development, quicker problem solving, and higher quality software delivery. What we see is that this shift is also helping collaborative initiatives like the Open Healthcare Network scale open source healthcare solutions across hundreds of hospitals in India.

Q. Where does GitHub stand today in the AI driven software development stack?

A. GitHub sees AI, agentic AI, and security as core parts of the developer workflow rather than standalone add-ons. Tools like Copilot and AI agents are integrated directly into workflows such as issue creation, pull requests, code reviews, CI pipelines, and debugging. GitHub is also enabling new AI native workflows through tools like Copilot CLI and the Copilot app, allowing developers to work more efficiently across terminals, web platforms, and orchestrated AI agent environments while maintaining flexibility and choice.

Q. What are the biggest structural changes in GitHub's developer ecosystem today?

A. One of the biggest changes at GitHub has been the shift from a version control platform to a complete AI powered software development ecosystem. GitHub expanded from code collaboration into automation through GitHub Actions, and now supports AI driven workflows across planning, building, testing, deployment, and operations. A major focus has been giving developers flexibility through model choice, agent choice, and workflow choice, allowing them to use AI models from OpenAI, Anthropic, Google, and others alongside tools like Copilot and Codex. Developers can now work seamlessly across web, mobile, CLI, VS Code, and other IDEs, making AI agents a core part of modern collaborative software development.

Q. How is debugging, architecture, and system design evolving in an AI assisted development environment?

A. Debugging, architecture, and system design are becoming more intelligent and collaborative in an AI assisted environment, where developer fundamentals remain critical for identifying what can go wrong and how to resolve it. At the same time, effective use of agentic AI allows developers to prevent and resolve issues more efficiently by clearly communicating system constraints and letting agents help design and build within those boundaries. With tools like Copilot integrated into platforms such as GitHub, agents can access repositories, observability systems, and error logs to correlate issues, trace root causes, and even propose fixes through pull requests. These changes enable a workflow where multiple specialised agents assist in debugging, code review, and architectural validation, while developers apply judgment to ensure system-level correctness and scalability.

Q. What differentiates successful open source projects from the thousands that never scale globally?

A. Successful open source projects are defined less by raw scale and more by how effectively they solve a specific problem for their intended users, whether niche or large scale. Key differentiators include clear problem fit, active maintenance that keeps the project updated for new features, performance needs, and evolving requirements, and a strong community that contributes, gives feedback, and helps others adopt and improve the project. Equally important is collaboration across developers, maintainers, organisations, and even other open source projects, since the ethos of openness and shared contribution drives long term success. Practices and guidance like those in open source guides also help maintainers improve project health and adoption.

Q. Are there any skill gaps despite India's massive developer base?

A. The focus is less on skill gaps and more on continuous upskilling as the industry evolves. A key requirement today is the effective use of AI and agentic AI tools, which are becoming central to developer productivity. At the same time, strong fundamentals in computer science, software engineering, and system design remain critical for building scalable and reliable systems. As applications increasingly need to work across low bandwidth environments, diverse user needs,